

HelixMemory

HelixMemory

Један меморијски мозак за агенте AI — четири врхунска мотора, спојена у једно.

Сажетак

HelixMemory је Go SDK који обједињује четири водећа меморијска система (Mem0, Cognee, Letta, Graphiti) у јединствени когнитивни меморијски мотор који их претражује паралелно и спаја резултате. Агенсима AI пружа један трајан, дедупликовани и прерангирани меморијски слој уместо четири неповезана.

Кратак опис

HelixMemory је Go SDK који спаја Mem0, Cognee, Letta и Graphiti у јединствени когнитивни меморијски мотор за апликације AI. Интелигентно усмерава уписе, паралелно претражује све позадинске системе и спаја резултате кроз тростепени ток прикупљања, дедупликације и прерангирања.

Детаљан опис

HelixMemory је јединствени когнитивни меморијски мотор за апликације AI, испоручен као Go SDK (модул `digital.vasic.helixmemory`, верзија Go 1.25+). Његова темељна претпоставка је да ниједан меморијски пројекат никада неће бити најбољи у свему — уместо да се меморија поново имплементира од нуле и наследи слепа места једног пројекта, он оркестрира четири врхунска система и омогућава сваком да игра на својим снагама: Mem0 за динамично издвајање чињеница и управљање преференцијама, Cognee за семантичке мреже знања изграђене кроз ЕЦЛ токове, Letta за стање агента са измењивим меморијским блоковима и

рачунањем током „сна“, и Graphiti за битемпоралну мрежу знања која расуђује о томе како се чињенице мењају кроз време.

Мотор за спајање је оно што те четири независне меморије претвара у један мозак. На путу уписа, свака нова меморија се класификује према садржају и усмерава ка позадинском систему који је за њу најпогоднији. На путу читања, упит се истовремено шаље свим позадинским системима, а добијени резултати улазе у тростепени ток спајања — прикупљање, дедупликација, а затим прерангирање на основу више извора — тако да позивалац никада не види четири бучна, преклапајућа скупа резултата, већ само један чист, рангирани одговор. Заштитни прекидачи обавијају сваки позадински систем како би омогућили поступно смањење перформанси: када један мотор откаже, његов прекидач се активира и преостали системи настављају да раде, уместо да повуку цео меморијски слој са собом. Пошто мотор имплементира интерфејс MemoryStore који се може директно заменити, он се уграђује као директна замена за обичног пружаоца меморије — без потребе за преуређењем позиваоца — а Prometheus метрике откривају унутрашње процесе усмеравања и спајања ради потпуне уочљивости.

HelixMemory је развијен као меморијски слој за HelixAgent, шири ансамбл агената Helix AI, и уноси породичну дисциплину тестирања без блефирања у домен меморије: уграђени изазивач тестира стварне продукционе токове — усмеравање, спајање, превод, заштитни прекидач — док упарени мутациони омотач намерно нарушава инваријанте како би доказао да тестови стварно падају када је логика покварена, тако да зелени сет значи нешто.

Зашто смо га направили

AI агентима потребна је дуговечна, висококвалитетна меморија, али је екосистем фрагментиран – сваки меморијски пројекат (Mem0, Cognee, Letta, Graphiti) снажан је у једној области, а слаб у другима. HelixMemory створен је како би HelixAgent агентима пружио јединствену меморијску површину која обједињује њихове предности, а да их притом не веже за било који од њих.

Зашто је ово револуционарно

Укида принудни избор. Четири меморијска система, који се иначе такмиче за исто место, постају комплементарни позадински системи иза једног јединственог интерфејса – тако да апликација добија динамично издвајање чињеница,

семантичке графове знања, меморију агента са стањем и битемпорално резонување *истовремено*, уз аутоматско уклањање дупликата и поновну рангираност на основу више извора. Оно што раније није било практично – третирати питање „који меморијски погон да усвојимо?“ као лажну дилему – HelixMemory омогућава да истовремено искористите све њихове предности, иза једног једноставног MemoryStore-a, без наслеђивања слабости ниједног погона или обавезе на закључавање.

Шта је иновативно

- **Фузија више позадинских система** (прикупљање → уклањање дупликата → поновна рангираност на основу више извора) која враћа један рангирани скуп резултата, уместо да позиваоца веже за један складиште.
- **Интелигентно усмеравање уписа** које класификује сваку меморију према садржају и шаље је погону најпогоднијем за њено чување, тако да прави подаци заврше у правом складишту.
- **Еlegantна деградација** путем прекидача по појединачним позадинским системима – отказивање једног погона није фатално, већ се изолује, а остали настављају да раде.
- **Консолидација током „времена спавања“** (путем Letta) која преуређује меморију током периода неактивности, уместо само у тренутку упита.
- **Провера против обмањивања:** покретач изазова над стварним продукцијским кодом, упарен са омотачем за мутације који мора да откаже када се наруши инваријанта – чиме се доказује да је тестна капија стварна провера, а не таутологија.

Највећи технички изазови и како смо их решили

- **Усклађивање четири хетерогена позадинска система у један кохерентан скуп резултата** – сваки погон враћа меморију у свом облику, а њихово наивно спајање доводи до дупликата и неупоредивих рангова. Решено помоћу типизованог фузионог погона који прикупља податке из више извора, уклања преклапања и све поново рангира на заједничкој основи, уз тврдњу о фузионисаном броју у тестовима како спајање не би тихо изгубило или двоструко рачунало резултате.
- **Одржавање рада када откаже позадински систем** – један недоступан меморијски погон не сме да заустави цео слој. Решено помоћу прекидача по појединачним

позадинским системима који прате машину стања затворено → отворено (након прага отказа) → полуотворено (након истека времена), изолирајући неисправан погон и настављајући да служи из оних који раде, све док се неисправан не опорави.

- **Доказивање да меморијска логика заиста функционише, а не само да се компајлира** – зелени сет тестова нема смисла ако тестови не могу да откажу. Решено помоћу покретача изазова у процесу који покреће стварни продукцијски код (усмеравање, фузију, преводилац, прекидач) и упареног омотача за мутације који мења инваријанте и захтева да тестови „поцрвене“, чиме се доказује да капија није таутологија.

Технолошки стек

- **Go (1.25+)** — јединствени SDK и извршно окружење; изабран јер паралелно читање са четири позадинска система представља проблем конкурентности, а горутине у Go-у то чине јефтиним, док његови интерфејсни типови читавом систему пружају једну чисту спојну тачку (MemoryStore) на коју се позивачи могу ослонити.
- **Mem0** — динамички бацкенд за издвајање чињеница и управљање преференцијама; користи се за сегмент меморије који одговара на питања „шта овај корисник заправо преферира / које су чињенице изрониле“.
- **Cognee** — семантички бацкенд заснован на графовима знања, изграђен на ЕЦЛ проточним системима; користи се за чување структурираног, повезаног знања уместо равних чињеница.
- **Letta** — бацкенд за извршно окружење агената са стањем, са могућношћу уређивања блокова меморије и рачунања током периода неактивности; користи се тамо где меморија мора да опстане као живо стање агента и да се консолидује током мировања.
- **Graphiti** — бацкенд за двовременски граф знања; користи се за расуђивање о томе како се чињенице и односи мењају током времена, а не само о њиховој тренутној вредности.
- **PostgreSQL + Neo4j + Redis** — стварне податковне базе на којима раде бацкендови, постављене за аутентично интеграционо тестирање путем `make infra-start`, како би тестни скупови користили стварну инфраструктуру уместо симулација.
- **Prometheus** — метрички подаци и могућност посматрања интегрисани кроз фузиони пипелине, тако да се понашање рутирања и фузије може мерити у продукцији, а не остаје црна кутија.
- **и18н спојни слој за превођење** — именовани (`helixmemory_`) низови знакова задржани на месту како би

сваки будући кориснички интерфејс могао да се локализује без накнадног прилагођавања језгра.

Статус и напомене о искрености

- **Статус: бета.** Функционалан SDK; изграђен као слој меморије за HelixAgent.
- **Лиценца: није одређена.** Није пронађена ЛИЦЕНЦА путем GitHub API — НЕВЕРИФИКОВАНО / није декларисано.
- Приказано име „HelixMemory” одговара репозиторијуму memory. Наведене вредности тачности у РЕАДМЕ фајлу преузете су од добављача, а нису мерене од стране HelixMemory, те су изостављене овде.

Приоритетни ниво: Helix-основни.